

PRUEBA TÉCNICA

Desarrollador(a) Full Stack — Nivel Semi Senior

Python · Django · DRF · Celery · RabbitMQ · PostgreSQL · React · JWT

Bienvenida

¡Gracias por tu interés en formar parte de nuestro equipo! Esta prueba busca conocer cómo abordan un problema real de desarrollo full stack, similar a los retos que enfrentarán en el día a día del equipo. No esperamos una solución perfecta ni de nivel productivo: nos interesa ver tu criterio técnico, cómo estructuras el código y cómo tomas decisiones.

1. Objetivo de la prueba

Deberás construir una funcionalidad end-to-end (backend + frontend) que simule parte de un sistema de gestión de pedidos (OMS) para un e-commerce, integrando el stack técnico de la posición.

2. Escenario

TiendaExpress es una empresa de e-commerce que necesita automatizar el flujo de sus pedidos: validar stock disponible, verificar el pago de forma asíncrona y notificar al cliente una vez confirmado. Tu tarea es implementar ese flujo, exponerlo mediante una API y consumirlo desde una interfaz web protegida con autenticación JWT.

3. Duración y modalidad de entrega

- *Formato:* prueba para realizar en casa (take-home).
- *Tiempo estimado de trabajo efectivo:* 8 horas.

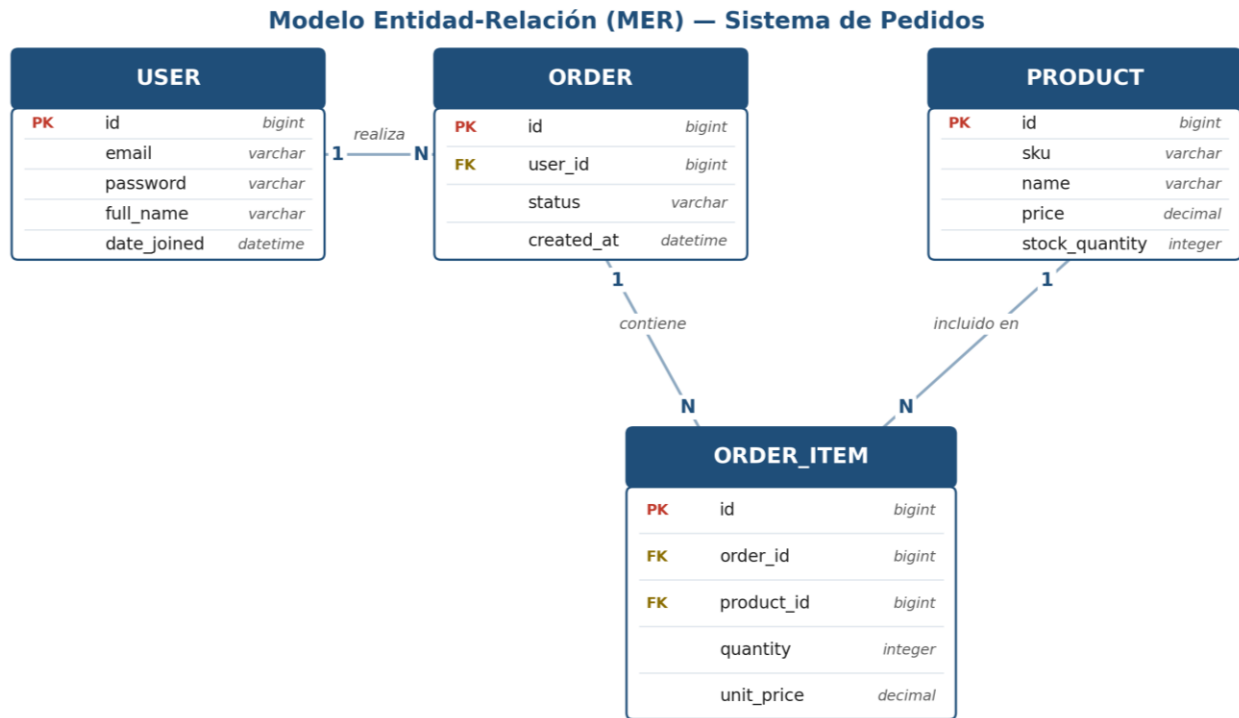
4. Stack técnico requerido

Backend	Frontend
Python 3.11+	React (Vite o CRA)
Django 4.x / 5.x	Axios o fetch
Django REST Framework	React Router
Celery + RabbitMQ	Manejo de JWT (access + refresh)
PostgreSQL	
djangoestframework-simplejwt	

Docker / Docker Compose es opcional, pero valorado si facilita la ejecución de tu solución.

5. Requerimientos — Backend

Modelos sugeridos (puedes ajustar nombres o campos si lo justificas)



Endpoints a implementar:

- POST /api/auth/login/ — obtiene access y refresh token
- POST /api/auth/refresh/
- GET /api/products/ — listado paginado
- POST /api/orders/ — crea un pedido, valida stock y dispara el procesamiento asíncrono
- GET /api/orders/ — listado, filtrable por status
- GET /api/orders/{id}/ — detalle con ítems y estado actual

Flujo asíncrono (Celery + RabbitMQ como broker):

- Al crear un pedido, este debe quedar en estado PENDING y disparar una tarea que simule la verificación de pago (por ejemplo, con un retardo y un resultado aleatorio o determinístico).
- Si la verificación es exitosa: descuenta el stock correspondiente, marca el pedido como CONFIRMED y dispara una segunda tarea que simule el envío de una notificación al cliente (puede ser un log o un email simulado).
- Si falla: marca el pedido como FAILED sin descontar stock.
- Maneja los errores de forma explícita, evita dejar un pedido en un estado inconsistente si una tarea falla.

Reglas de negocio:

- No se debe permitir crear un pedido si el stock es insuficiente.

- *Extra opcional: evitar sobreventa (oversell) ante creación concurrente de pedidos, por ejemplo con `select_for_update()`.*

6. Requerimientos — Frontend

- Login: pantalla que autentica contra `/api/auth/login/` y almacena el JWT.
- Manejo de JWT: interceptor (Axios o equivalente) que adjunte el access token a cada request y maneje el refresh automático cuando expire.
- Listado de pedidos: consume `/api/orders/`, muestra el estado de cada uno con un indicador visual, y permite refrescar o consultar el estado actualizado (recuerda que la confirmación es asíncrona).
- Formulario de creación de pedido: selección de producto(s) y cantidad, envío a `/api/orders/`, con manejo de errores (stock insuficiente, sesión expirada, etc.).
- Rutas protegidas: redirigir a login si no hay una sesión válida.

7. Entregables

- Repositorio con el código de backend y frontend (Git).
- README con instrucciones claras para levantar el proyecto (idealmente con `docker-compose up`).
- Archivo `.env.example` con las variables necesarias.
- Una breve sección explicando tus decisiones de diseño: qué priorizaste y qué harías distinto con más tiempo.

8. Cómo se evaluará tu prueba

A continuación compartimos los criterios y pesos que utilizaremos para revisar tu entrega, de forma que puedas priorizar tu tiempo:

Criterio	Peso
Diseño de modelos y API REST (convenciones, serializers, validaciones)	20%
Implementación de Celery + RabbitMQ (tareas asíncronas, manejo de errores/estados)	20%
Uso correcto de PostgreSQL / transacciones	10%
Autenticación JWT (backend y frontend, incluyendo refresh)	20%
Frontend: manejo de estado, UX, manejo de errores de red o sesión expirada	15%
Calidad de código y estructura (legibilidad, separación de responsabilidades)	10%
Documentación (README, decisiones de diseño)	5%

9. Puntos extra (opcionales)

No son obligatorios, pero suman valor a tu entrega si te sobra tiempo:

- Tests unitarios (pytest / DRF test client).
- Docker Compose funcional de punta a punta.
- Manejo de concurrencia (`select_for_update`, constraints de base de datos).
- Polling o WebSocket para reflejar el cambio de estado del pedido en tiempo real en el frontend.

- Paginación y filtros avanzados en el listado de pedidos.

10. Instrucciones de entrega

- Sube tu solución a un repositorio (GitHub, GitLab o similar) y compártenos el enlace del proyecto.
- Incluye el README con los pasos para ejecutar tu solución localmente.
- Ante cualquier duda sobre el alcance o los requerimientos, no dudes en contactarnos preferimos que preguntes a que asumas algo incorrecto.